

untangle::async

Generated by Doxygen 1.18.0

1	async	1
2	Topic Index	5
2.1	Topics	5
3	Namespace Index	7
3.1	Namespace List	7
4	Class Index	9
4.1	Class List	9
5	File Index	11
5.1	File List	11
6	Topic Documentation	13
6.1	namespace untangle: functions	13
6.1.1	Detailed Description	13
6.1.2	Function Documentation	13
6.1.2.1	bind() [1/2]	13
6.1.2.2	bind() [2/2]	14
7	Namespace Documentation	15
7.1	untangle Namespace Reference	15
7.1.1	Detailed Description	15
8	Class Documentation	17
8.1	untangle::async::execution< actionT > Class Template Reference	17
8.1.1	Detailed Description	17
8.1.2	Constructor & Destructor Documentation	18
8.1.2.1	execution()	18
8.1.2.2	~execution()	18
8.1.3	Member Function Documentation	18
8.1.3.1	bind_action_and_method()	18
8.1.3.2	is_running()	19
8.1.3.3	stop()	19
8.1.3.4	add_action()	19
8.2	untangle::async::execution_poll Class Reference	20
8.2.1	Detailed Description	20
8.2.2	Member Function Documentation	20
8.2.2.1	add()	20
8.2.2.2	remove()	21
8.2.2.3	is_running()	21
8.2.2.4	get()	21
9	File Documentation	23
9.1	async.hpp	23

Chapter 1

async

c++ mechanism to run queued callables in an asynchronous fashion

The interface consists of one header file that exposes a generic execution class. It is a wrapper of a list of callables that are invoked on other thread. The callables are pumped into the list by invoking an action from the caller thread.

The implementation is using the **actuator** callable, and therefore it exemplifies how a generic callable may improve the code structure, by easily creating interfaces inside a class instead of using external ones.

There are provided two execution modes: one-off and continuous.

requirements

C++23. The interface reports warnings with `std::println`, so it needs a standard library that provides `<print>` — GCC 14 or newer; older `libc++` and `MSVC` releases may not have it yet.

one-off execution



example: one-off execution of a plain function

```
#include <async.hpp>

void f()
{
    std::cout << "f thread " << std::this_thread::get_id() << std::endl;
}

int main()
{
    std::cout << "main thread " << std::this_thread::get_id() << std::endl;

    // declare an execution specialized for an action type
    // the name is optional, and identifies the execution in the warnings it reports
    untangle::async::execution<std::function<void(void)>> execution{"oneoff"};
    std::function<void(void)> action;
    // create async binding between the action and f
```

```

execution.bind_action_and_function(action, f);
// whenever the action is invoked, a new callable wrapping f will be added to the execution's action list
action();
// run the execution
execution.run();

// add the execution object to the running poll
untangle::async::execution_poll::get().add(execution);

// wait the polled executions to finish
while(untangle::async::execution_poll::get().is_running())
{
    std::this_thread::sleep_for(std::chrono::milliseconds(1));
}

return 0;
}

```

example: one-off execution of a class method

```

#include <async.hpp>

class A
{
public:
    void f(int x)
    {
        std::cout << "A::f " << x << " thread " << std::this_thread::get_id() << std::endl;
    }
    std::function<void(int)> action;
};

int main()
{
    std::cout << "main thread " << std::this_thread::get_id() << std::endl;

    auto a = std::make_shared<A>();

    // declare an execution specialized for an action type
    untangle::async::execution<std::function<void(int)>> execution{"oneoff_method"};
    execution.bind_action_and_method(a->action, a, &A::f); // note the bound object must be a shared pointer
    // whenever the action is invoked, a new callable wrapping f will be added to the execution's action list
    a->action(10);
    // run the execution
    execution.run();

    // add the execution object to the running poll
    untangle::async::execution_poll::get().add(execution);

    // wait the polled executions to finish
    while(untangle::async::execution_poll::get().is_running())
    {
        std::this_thread::sleep_for(std::chrono::milliseconds(1));
    }

    return 0;
}

```

continuous execution



example: continuous execution of a plain function

```

#include <async.hpp>

void f()
{
    std::cout << "f thread " << std::this_thread::get_id() << std::endl;
}

int main()
{

```

```

std::cout << "main thread " << std::this_thread::get_id() << std::endl;

// declare an execution specialized for an action type
untangle::async::execution<std::function<void(void)>> execution{"continuous"};
std::function<void(void)> action;
// create async binding between the action and f
execution.bind_action_and_function(action, f);

// run the execution
execution.start();
// add f to the execution's queue (three times)
action();action();action();
// stop the execution
execution.stop();

// add the execution object to the running poll
untangle::async::execution_poll::get().add(execution);

// wait the polled executions to finish
while(untangle::async::execution_poll::get().is_running())
{
    std::this_thread::sleep_for(std::chrono::milliseconds(1));
}

return 0;
}

```

queueing and lifetime

Queueing is thread safe. An action may be invoked from any thread; the execution's list is guarded, and the worker is woken rather than polled for.

`stop()` ends the execution's working life. The worker runs whatever was queued before the call and then leaves. An action invoked after `stop()` is refused rather than queued, because it would never run, and the refusal is reported:

```
warning: execution 'continuous' is stopped, action not added
```

An action bound to an object that has since been destroyed is dropped the same way, rather than ending the process:

```
warning: execution 'continuous' dropped an invalid action: bind: invalid object
```

The worker is detached and cannot be joined. Waiting for an execution therefore means waiting on its running state, which is what `execution_poll` reports and what the examples above do. The destructor waits on the same state, so an execution that goes out of scope while its worker is still running blocks until the worker is finished rather than leaving it reading freed memory.

The poll holds a pointer to each execution added to it, so an execution withdraws itself in its destructor. `execution_poll::remove()` is available for withdrawing one earlier; calling it is not required.

Chapter 2

Topic Index

2.1 Topics

Here is a list of all topics with brief descriptions:

namespace untangle: functions [13](#)

Chapter 3

Namespace Index

3.1 Namespace List

Here is a list of all documented namespaces with brief descriptions:

untangle	Interface to untangle::async::execution class	15
--------------------------	---	--------------------

Chapter 4

Class Index

4.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

untangle::async::execution< actionT >	
Asynchronous template execution class	17
untangle::async::execution_poll	
Execution poll class	20

Chapter 5

File Index

5.1 File List

Here is a list of all documented files with brief descriptions:

async.hpp		23
---------------------------	-------	----

Chapter 6

Topic Documentation

6.1 namespace untangle: functions

Functions

- `template<typename classT, typename T, typename actionT = std::function<T>>>`
`actionT untangle::async::bind (const std::shared_ptr< classT > &obj, T classT::*method,`
`execution< actionT > &async_exec)`
Asynchronous binding to a class function member.
- `template<typename T, typename actionT = std::function<T>>>`
`actionT untangle::async::bind (T &Fn, execution< actionT > &async_exec)`
Asynchronous binding to a plain function.

6.1.1 Detailed Description

6.1.2 Function Documentation

6.1.2.1 `bind()` [1/2]

```
template<typename classT, typename T, typename actionT = std::function<T>>>
actionT untangle::async::bind (
    const std::shared_ptr< classT > & obj,
    T classT::* method,
    execution< actionT > & async_exec)
```

Asynchronous binding to a class function member.

It uses an [execution](#) object to execute the bound class function member on other thread. It creates an action as a binding to a class method (by `untangle::bind()`), and returns a callable that passes this action to [execution::add_action\(\)](#).

Parameters

<code>obj</code>	- A <code>std::shared_ptr</code> that wraps the bound class object.
------------------	---

method	- Pointer to function member. It is specified as &<class type>::<function member>.
async_exec	- An execution object.

Returns

- A `std::function<...>(lambda)` that adds the action to the execution object's actions list.

6.1.2.2 `bind()` [2/2]

```
template<typename T, typename actionT = std::function<T>>>
actionT untangle::async::bind (
    T & Fn,
    execution< actionT > & async_exec)
```

Asynchronous binding to a plain function.

It uses an [execution](#) object to execute the bound function on other thread. It creates an action as `std::function` that wraps the function, and returns a callable that passes this action to [execution::add_action\(\)](#).

Parameters

Fn	- A plain function.
async_exec	- An execution object.

Returns

- A `std::function(lambda)` that adds the action to the execution object's actions list.

Chapter 7

Namespace Documentation

7.1 untangle Namespace Reference

Interface to [untangle::async::execution](#) class.

7.1.1 Detailed Description

Interface to [untangle::async::execution](#) class.

Chapter 8

Class Documentation

8.1 `untangle::async::execution< actionT >` Class Template Reference

Asynchronous template execution class.

```
#include <async.hpp>
```

Public Member Functions

- `execution ()`
Constructs a new execution object.
- `execution (std::string exec_name)`
Constructs a new named execution object.
- `~execution ()`
Destroys the execution object, once its worker has left.
- `template<typename classT, typename T>`
`void bind_action_and_method (actionT &action, const std::shared_ptr< classT > &obj, T class↵
T::*method)`
Binds asynchronously an external action to a class function member.
- `bool is_running () const`
Checks if this execution has finished.
- `void stop ()`
Stops the worker, after the actions already queued have run.
- `template<typename... Args>`
`void add_action (actionT action, Args... args)`

Static Public Attributes

- `static constexpr auto default_name = "unnamed"`
Name of an execution built unnamed.

8.1.1 Detailed Description

```
template<typename actionT>  
class untangle::async::execution< actionT >  
Generated by Doxygen
```

Asynchronous template execution class.

A `untangle::async::execution` will be used to execute actions asynchronously. It will execute all the actions in the queue and then return the result of the last action.

Template Parameters

actionT	It represents the type of the action. It is specified as <code>std::function<...></code> and should match the signature of the bound function or class method.
---------	--

8.1.2 Constructor & Destructor Documentation

8.1.2.1 execution()

```
template<typename actionT>
untangle::async::execution< actionT >::execution (
    std::string exec_name) [inline], [explicit]
```

Constructs a new named execution object.

The name identifies this execution in the warnings it reports, and is otherwise unused. An execution built by the default constructor keeps the default name rather than an empty one, so a warning always names something.

Parameters

exec_name	- A name for this execution.
-----------	------------------------------

8.1.2.2 ~execution()

```
template<typename actionT>
untangle::async::execution< actionT >::~~execution () [inline]
```

Destroys the execution object, once its worker has left.

The worker is detached and cannot be joined, so this waits on the same "running" state that [execution_poll](#) reports - the only handle a detached worker offers. It returns at once for an execution that was never started, or that the caller has already polled to a stop.

Remarks

This is safe only because running is cleared as the very last thing the worker does. Nothing may be added after it in `execute()` or `loop()`: the object can be freed the moment it reads false.

8.1.3 Member Function Documentation

8.1.3.1 bind_action_and_method()

```
template<typename actionT>
template<typename classT, typename T>
void untangle::async::execution< actionT >::bind_action_and_method (
    actionT & action,
    const std::shared_ptr< classT > & obj,
    T classT::* method) [inline]
```

Parameters

action	[in,out] - An action of type <code>std::function<...></code> .
obj	- A <code>std::shared_ptr</code> that wraps the bound class object.
method	- Pointer to function member. It is specified as <code>&<class type>::<function member></code> .

8.1.3.2 `is_running()`

```
template<typename actionT>
bool untangle::async::execution< actionT >::is_running () const [inline]
```

Checks if this execution has finished.

Returns

`true` - The execution has not finished.
`false` - The execution has finished.

8.1.3.3 `stop()`

```
template<typename actionT>
void untangle::async::execution< actionT >::stop () [inline]
```

Stops the worker, after the actions already queued have run.

Remarks

An action passed to `add_action()` after this call is refused, not queued: `stop()` is the end of this execution's working life, and an action accepted then would never run.

8.1.3.4 `add_action()`

```
template<typename actionT>
template<typename... Args>
void untangle::async::execution< actionT >::add_action (
    actionT action,
    Args... args) [inline]
```

Template Parameters

Args	
------	--

Parameters

action	
args	

The documentation for this class was generated from the following file:

- `async.hpp`

8.2 untangle::async::execution_poll Class Reference

Execution poll class.

```
#include <async.hpp>
```

Public Member Functions

- `template<typename asyncexecT>`
`void add (asyncexecT &async_exec)`
 Adds an `execution` object to the poll.
- `template<typename asyncexecT>`
`void remove (asyncexecT &async_exec)`
 Removes an `execution` object from the poll.
- `auto is_running ()`
 Checks if the execution objects are running.

Static Public Member Functions

- `static execution_poll & get ()`
 Gets the single-tone instance.

8.2.1 Detailed Description

Execution poll class.

A single-tone class that may be used to verify if the polled `execution` objects have finished their processing. If at least one `execution` object is running then this poll has the "running" state.

Remarks

An `execution` object runs in a detached thread, hence it is not joinable. The caller can wait for an `execution` to be finished only by checking the "running" state(`execution::is_running()`). This class represents a convenient way to check any number of `execution` objects.

8.2.2 Member Function Documentation

8.2.2.1 add()

```
template<typename asyncexecT>
```


Parameters

async_exec	An execution object.
------------	--------------------------------------

8.2.2.2 remove()

```
template<typename asyncexecT>
void untangle::async::execution_poll::remove (
    asyncexecT & async_exec) [inline]
```

Removes an [execution](#) object from the poll.

[add\(\)](#) stores a pointer to the object's `action_is_running`, so an execution destroyed while still registered leaves the poll calling into freed memory. `~execution()` calls this.

Parameters

async_exec	An execution object.
------------	--------------------------------------

8.2.2.3 is_running()

```
auto untangle::async::execution_poll::is_running () [inline]
```

Checks if the execution objects are running.

Returns

true - if at least one [execution](#) object in this poll is running.

8.2.2.4 get()

```
execution_poll & untangle::async::execution_poll::get () [inline], [static]
```

Gets the single-tone instance.

Returns

[execution_poll](#) An [execution_poll](#) instance.

The documentation for this class was generated from the following file:

- `async.hpp`

Chapter 9

File Documentation

9.1 async.hpp

```
00001 // Copyright (c) 2018 Nicolae Popescu. MIT License.
00002
00006 #pragma once
00007
00008 #include <actuator/actuator.hpp>
00009 #include <atomic>
00010 #include <condition_variable>
00011 #include <mutex>
00012 #include <print>
00013 #include <string>
00014 #include <thread>
00015
00016 namespace untangle {
00017 template <typename actionT>
00018 struct actuator;
00019
00020 namespace async {
00021 template <typename actionT>
00022 class execution;
00023 }
00024 } // namespace untangle
00025
00026 namespace untangle {
00027 namespace async {
00031
00048 template <typename classT, typename T, typename actionT = std::function<T>»
00049 actionT bind(const std::shared_ptr<classT>& obj, T classT::* method,
00050             execution<actionT>& async_exec) {
00051     actionT async_action = untangle::bind(obj, method);
00052
00053     return [&async_exec, async_action](auto... args) -> typename actionT::result_type {
00054         async_exec.add_action(async_action, args...);
00055         return typename actionT::result_type();
00056     };
00057 }
00058
00073 template <typename T, typename actionT = std::function<T>»
00074 actionT bind(T& Fn, execution<actionT>& async_exec) {
00075     actionT async_action = Fn;
00076     return [&async_exec, async_action](auto... args) -> typename actionT::result_type {
00077         async_exec.add_action(async_action, args...);
00078         return typename actionT::result_type();
00079     };
00080 }
00081
00096 class execution_poll {
00097 public:
00103     template <typename asyncexecT>
00104     void add(asyncexecT& async_exec) {
00105         if (!actuator_is_running.is_connected()) {
00106             actuator_is_running = untangle::connect(async_exec.action_is_running);
00107         } else {
00108             actuator_is_running.add(&async_exec.action_is_running);
00109         }
00110     }
00111
00120     template <typename asyncexecT>
```

```

00121 void remove(asyncexecT& async_exec) {
00122     actuator_is_running.remove(&async_exec.action_is_running);
00123 }
00124
00130 auto is_running() {
00131     actuator_is_running();
00132
00133     auto result = false;
00134     for (const auto& ret : actuator_is_running.results) {
00135         result |= ret;
00136     }
00137     return result;
00138 }
00139
00145 static execution_poll& get() {
00146     static execution_poll instance;
00147     return instance;
00148 }
00149
00150 private:
00151 execution_poll() = default;
00152 ~execution_poll() = default;
00153 actuator<std::function<bool(void)>> actuator_is_running;
00154 };
00155
00168 template <typename actionT>
00169 class execution {
00170 public:
00171     execution() {
00172         other_this = this;
00173         action_execute = untangle::bind(other_this, &execution<actionT>::execute_actions);
00174         action_stop = untangle::bind(other_this, &execution<actionT>::stop);
00175         action_is_running = untangle::bind(other_this, &execution<actionT>::is_running);
00176     }
00177
00181 explicit execution(std::string exec_name) : execution() { name = std::move(exec_name); }
00182
00183 ~execution() {
00184     {
00185         std::lock_guard<std::mutex> lock(action_mutex);
00186         started = false;
00187         stopped = true;
00188     }
00189
00190     action_cv.notify_all();
00191
00192     while (running.load()) {
00193         std::this_thread::sleep_for(std::chrono::milliseconds(1));
00194     }
00195
00196     execution_poll::get().remove(*this);
00197 }
00198
00200 template <typename classT, typename T>
00201 void bind_action_and_method(actionT& action, const std::shared_ptr<classT>& obj,
00202                             T classT::* method) {
00203     action = bind(obj, method, *this);
00204 }
00205
00206 template <typename T>
00207 void bind_action_and_function(actionT& action, const T& Fn) {
00208     action = bind(Fn, *this);
00209 }
00210
00211 bool is_running() const { return running.load(); }
00212
00213 void run() {
00214     running = true;
00215     this_thread = std::thread(&execution::execute, this);
00216     this_thread.detach();
00217 }
00218
00219 void start() {
00220     running = true;
00221     {
00222         std::lock_guard<std::mutex> lock(action_mutex);
00223         started = true;
00224         stopped = false;
00225     }
00226
00227     this_thread = std::thread(&execution::loop, this);
00228     this_thread.detach();
00229 }
00230
00231 void stop() {
00232     {
00233         std::lock_guard<std::mutex> lock(action_mutex);
00234         started = false;

```

```

00276     stopped = true;
00277 }
00278
00279 // The worker drains what is already queued before it leaves loop(), so stop() does not have to
00280 // spin on the list; waking it is enough.
00281 action_cv.notify_all();
00282
00283 if (actuator_stop.is_connected()) {
00284     actuator_stop();
00285 }
00286 }
00287
00295 template <typename... Args>
00296 void add_action(actionT action, Args... args) {
00297     {
00298         std::lock_guard<std::mutex> lock(action_mutex);
00299
00300         // Once stopped, the worker is on its way out and would never reach this action; dropping it
00301         // here is what keeps it from sitting in the list looking as though it were pending.
00302         if (stopped) {
00303             std::println(stderr, "warning: execution '{}' is stopped, action not added", name);
00304             return;
00305         }
00306
00307         action_list.push_back(std::bind(action, args...));
00308     }
00309
00310     action_cv.notify_one();
00311 }
00312
00313 template <typename otherT>
00314 void attach(otherT& other) {
00315     if (!actuator_execute.is_connected()) {
00316         actuator_execute = untangle::connect(other.action_execute);
00317     } else {
00318         actuator_execute.add(&other.action_execute);
00319     }
00320
00321     if (!actuator_stop.is_connected()) {
00322         actuator_stop = untangle::connect(other.action_stop);
00323     } else {
00324         actuator_stop.add(&other.action_stop);
00325     }
00326 }
00327
00328 auto result() { return _result; }
00329
00330 static constexpr auto default_name = "unnamed";
00331
00332 std::function<void(void)> action_execute;
00333 std::function<void(void)> action_stop;
00334 std::function<bool(void)> action_is_running;
00335 std::function<void(void)> on_finished;
00336 std::string name = default_name;
00337
00338 private:
00339 using queued_action_t = std::function<typename actionT::result_type(void)>;
00340
00341 void execute_action(queued_action_t& action) {
00342     if constexpr (std::is_void_v<typename actionT::result_type>) {
00343         action();
00344     } else {
00345         _result = action();
00346     }
00347 }
00348
00349 void execute_actions() {
00350     // The action is taken off the list under the lock and invoked with the lock released: an action
00351     // is caller code that may run for a while, and may itself call add_action().
00352     for (;;) {
00353         queued_action_t action;
00354
00355         {
00356             std::lock_guard<std::mutex> lock(action_mutex);
00357             if (action_list.empty()) {
00358                 break;
00359             }
00360
00361             action = std::move(action_list.front());
00362             action_list.pop_front();
00363         }
00364
00365         // An action bound to an object that has since died throws invalid_action. Letting it leave a
00366         // thread function calls std::terminate, and a detached worker gives no one a chance to catch
00367         // it; the actuator drops such an action, and so does this.
00368         try {
00369             execute_action(action);
00370         }
00371     }

```

```

00373     } catch (const untangle::invalid_action& ia) {
00374         std::println(stderr, "warning: execution '{}', dropped an invalid action: {}", name,
00375             ia.what());
00376     }
00377 }
00378
00379 if (actuator_execute.is_connected()) {
00380     actuator_execute();
00381 }
00382 }
00383
00384 void execute() {
00385     execute_actions();
00386
00387     if (on_finished) {
00388         on_finished();
00389     }
00390
00391     std::cout << "finishing thread" << std::endl;
00392
00393     running = false;
00394 }
00395
00396 void loop() {
00397     for (;;) {
00398         {
00399             std::unique_lock<std::mutex> lock(action_mutex);
00400
00401             // A bounded wait rather than a plain one: an attached execution has its own list and no way
00402             // to notify this condition variable, so the worker still has to look in on it periodically.
00403             action_cv.wait_for(lock, std::chrono::milliseconds(10),
00404                 [this] { return !action_list.empty() || !started; });
00405
00406             if (!started && action_list.empty()) {
00407                 break;
00408             }
00409         }
00410
00411         execute_actions();
00412     }
00413
00414     // What was queued before stop() still belongs to this execution; nothing can have been added
00415     // after it, because add_action() refuses once stopped.
00416     execute_actions();
00417
00418     std::cout << "thread finished" << std::endl;
00419
00420     // Must stay last: ~execution() may free this object the moment it reads false.
00421     running = false;
00422 }
00423
00424 std::thread this_thread;
00425
00426 std::atomic_bool started = {false};
00427 std::atomic_bool running = {false};
00428
00429 // Set by stop(), and the difference between "not started yet" and "finished for good": actions
00430 // may be queued before run()/start(), but not after stop().
00431 std::atomic_bool stopped = {false};
00432
00433 // action_list, started and stopped are written by every thread that calls add_action() or stop()
00434 // and read by the worker; nothing touches them outside this mutex. action_cv is what replaced the
00435 // spin in loop() and in stop().
00436 mutable std::mutex action_mutex;
00437 std::condition_variable action_cv;
00438
00439 std::list<std::function<typename actionT::result_type(void)>> action_list;
00440
00441 actuator<std::function<void(void)>> actuator_execute;
00442 actuator<std::function<void(void)>> actuator_stop;
00443
00444 // std::vector cannot hold void type; use an arbitrary type e.g. int
00445 using resultT = std::conditional<std::is_void<typename actionT::result_type>::value, int,
00446     typename actionT::result_type>;
00447 typename resultT::type _result;
00448
00449 execution<actionT>* other_this;
00450 };
00451 } // namespace async
00452 } // namespace untangle

```

Index

- ~execution
 - untangle::async::execution< actionT >, [18](#)
- add
 - untangle::async::execution_poll, [20](#)
- add_action
 - untangle::async::execution< actionT >, [19](#)
- async, [1](#)
- bind
 - namespace untangle: functions, [13](#), [14](#)
- bind_action_and_method
 - untangle::async::execution< actionT >, [18](#)
- execution
 - untangle::async::execution< actionT >, [18](#)
- get
 - untangle::async::execution_poll, [21](#)
- is_running
 - untangle::async::execution< actionT >, [19](#)
 - untangle::async::execution_poll, [21](#)
- namespace untangle: functions, [13](#)
 - bind, [13](#), [14](#)
- remove
 - untangle::async::execution_poll, [21](#)
- stop
 - untangle::async::execution< actionT >, [19](#)
- untangle, [15](#)
- untangle::async::execution< actionT >, [17](#)
 - ~execution, [18](#)
 - add_action, [19](#)
 - bind_action_and_method, [18](#)
 - execution, [18](#)
 - is_running, [19](#)
 - stop, [19](#)
- untangle::async::execution_poll, [20](#)
 - add, [20](#)
 - get, [21](#)
 - is_running, [21](#)
 - remove, [21](#)